Übung 1 · Musterlösungen

Algorithmische Grundbausteine
Programmierung 1 · WS 2026/27 · André Kless, M.Sc.

Die folgenden Lösungen zeigen mögliche Lösungswege. Andere Algorithmen sind ebenfalls richtig, wenn sie für alle zulässigen Eingaben die geforderte Ausgabe liefern. Deklarationen erfolgen ohne Typangaben. Die Lösungen setzen die im Aufgabenblatt genannten Eingabebereiche voraus.

In den Struktogrammen bilden aneinandergefügte Blöcke eine Sequenz. Schleifen umschließen ihren Rumpf; Verzweigungen zeigen die alternativen Abläufe. In Trace-Tabellen bezeichnet „nicht belegt“ eine deklarierte Variable, die noch keinen Wert erhalten hat.

01 Kosten für die Lerngruppe

Eingabe: positive Anzahl Hefte. **Verarbeitung:** Anzahl mit 3 multiplizieren und einmalig 4 Euro addieren. **Ausgabe:** Gesamtpreis in Euro. Benötigt werden `anzahl` und `gesamt`.

Pseudocode

```
DEKLARIERE anzahl, gesamt
LIES anzahl EIN
SETZE gesamt AUF anzahl * 3 + 4
GIB gesamt AUS
```

Struktogramm

`DEKLARIERE anzahl, gesamt`
`LIES anzahl EIN`
`SETZE gesamt AUF anzahl * 3 + 4`
`GIB gesamt AUS`

Prüffälle

anzahl	Rechnung	Ausgabe
1	1 * 3 + 4	7 Euro
5	5 * 3 + 4	19 Euro

Die Lieferkosten fallen einmal pro Bestellung an. Sie stehen daher außerhalb der Multiplikation mit der Anzahl.

02 Zwei Werte vertauschen

a) Den fehlerhaften Versuch verfolgen

Anweisung / Zustand	a	b
Ausgangszustand	4	9
SETZE a AUF b	9	9
SETZE b AUF a	9	9

Die erste Zuweisung überschreibt den ursprünglichen Wert von a. Die zweite liest bereits den neuen Wert 9. Der Wert 4 steht dann in keiner Variable mehr zur Verfügung.

b) Korrekte Lösung mit Hilfsvariable

```
DEKLARIERE a, b, hilfe
LIES a EIN
LIES b EIN
SETZE hilfe AUF a
SETZE a AUF b
SETZE b AUF hilfe
GIB a, b AUS
```

Anweisung / Zustand	a	b	hilfe
Nach den Eingaben	4	9	nicht belegt
SETZE hilfe AUF a	4	9	4
SETZE a AUF b	9	9	4
SETZE b AUF hilfe	9	4	4

Ausgabe: 9 und 4. Die Hilfsvariable bewahrt den alten Wert von a, bis er nach b geschrieben wurde.

c) Gleiche Werte

Für a = 5 und b = 5 enthält nach jeder Zuweisung auch die Hilfsvariable den Wert 5. Die Ausgabe ist 5 und 5. Das Vertauschen ist korrekt, auch wenn keine sichtbare Änderung entsteht.

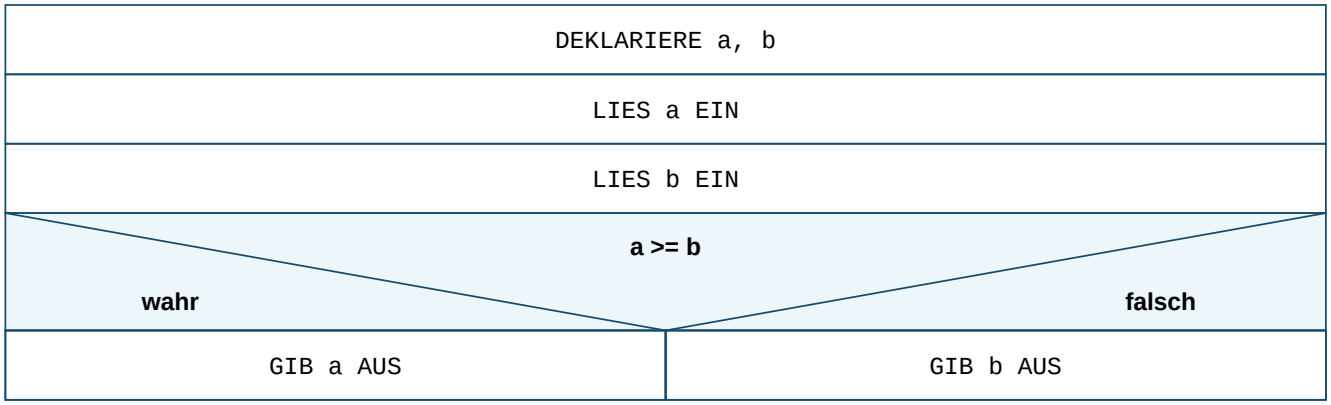
03 Die größere Zahl

Für a > b ist die Ausgabe a, für a < b ist sie b. Bei Gleichheit darf einer der beiden identischen Werte ausgegeben werden. Die Bedingung a >= b deckt den Gleichheitsfall bereits ab.

Pseudocode

```
DEKLARIERE a, b
LIES a EIN
LIES b EIN
WENN a >= b:
    GIB a AUS
SONST:
    GIB b AUS
ENDE WENN
```

Struktogramm



Prüffälle

Eingabe (a, b)	a >= b	Ausgabe	Abgedeckter Fall
(7, 3)	wahr	7	Erste Zahl größer
(3, 7)	falsch	7	Zweite Zahl größer
(5, 5)	wahr	5	Gleiche Werte
(-2, -8)	wahr	-2	Negative Werte

Nur ein Zweig läuft. Deshalb erfolgt auch bei Gleichheit genau eine Ausgabe.

04 Ein Menü für den Lernraum

Pseudocode

```
DEKLARIERE auswahl
LIES auswahl EIN
FALLS auswahl:
    BEI 1:
        GIB "Lernraum buchen" AUS
    BEI 2:
        GIB "Buchung anzeigen" AUS
    BEI 3:
        GIB "Buchung stornieren" AUS
    SONST:
        GIB "Ungültige Auswahl" AUS
ENDE FALLS
```

Struktogramm

<table>
<tr><td colspan="4">DEKLARIERE auswahl</td></tr>
<tr><td colspan="4">LIES auswahl EIN</td></tr>
<tr><td colspan="4">FALLS auswahl</td></tr>
<tr><td>1</td><td>2</td><td>3</td><td>SONST</td></tr>
<tr><td>GIB "Lernraum buchen" AUS</td><td>GIB "Buchung anzeigen" AUS</td><td>GIB "Buchung stornieren" AUS</td><td>GIB "Ungültige Auswahl" AUS</td></tr>
</table>

Die Mehrfachselektion führt genau einen Fall aus. SONST umfasst alle ganzen Zahlen außer 1, 2 und 3. Die Struktogramm-Darstellung stellt die vier Fälle nebeneinander.

Prüffälle

Eingabe	Ausgabe
1	„Lernraum buchen“
2	„Buchung anzeigen“
3	„Buchung stornieren“
0	„Ungültige Auswahl“
99	„Ungültige Auswahl“

05 Zahlen von 1 bis n addieren

`zaehler` enthält die nächste zu addierende Zahl. Er startet bei 1. `summe` startet bei 0, dem Ergebnis vor der ersten Addition.

a) Lösung mit SOLANGE

```
DEKLARIERE n, zaehler, summe
LIES n EIN
SETZE zaehler AUF 1
SETZE summe AUF 0
SOLANGE zaehler <= n:
    SETZE summe AUF summe + zaehler
    SETZE zaehler AUF zaehler + 1
ENDE SOLANGE
GIB summe AUS
```

Struktogramm

`DEKLARIERE n, zaehler, summe`
`LIES n EIN`
`SETZE zaehler AUF 1`
`SETZE summe AUF 0`
`SOLANGE zaehler <= n`
`SETZE summe AUF summe + zaehler`
`SETZE zaehler AUF zaehler + 1`
`GIB summe AUS`

Die Bedingung lautet `zaehler <= n`, damit n noch mitaddiert wird. Danach erhöht sich der Zähler auf n + 1 und die Schleife endet.

05 Zahlen von 1 bis n addieren (Fortsetzung)

b) Trace-Tabelle für n = 4

Jede Zeile zeigt die Werte unmittelbar vor der Bedingungsprüfung. Auch die letzte, falsche Prüfung gehört zur Tabelle.

Prüfung	zaehler	summe	zaehler <= n
1	1	0	wahr
2	2	1	wahr
3	3	3	wahr
4	4	6	wahr
5	5	10	falsch

Ausgabe: 10. Die aufaddierten Werte sind 1, 2, 3 und 4.

c) Grenzfälle

n	Erste Prüfung	Durchläufe	Ausgabe
0	1 <= 0: falsch	0	0
1	1 <= 1: wahr	1	1

Für n = 0 gibt es keine Zahl von 1 bis n. Die Summe bleibt deshalb bei ihrem Anfangswert 0.

d) Lösung mit WIEDERHOLE n-MAL

```
DEKLARIERE n, zaehler, summe
LIES n EIN
SETZE zaehler AUF 1
SETZE summe AUF 0
WIEDERHOLE n-MAL:
    SETZE summe AUF summe + zaehler
    SETZE zaehler AUF zaehler + 1
ENDE WIEDERHOLUNG
GIB summe AUS
```

In jedem Durchlauf muss der Algorithmus die aktuelle Zahl addieren und den Zähler erhöhen. Bei n = 0 wird der Rumpf keinmal ausgeführt. Beide Varianten liefern dieselben Ergebnisse.

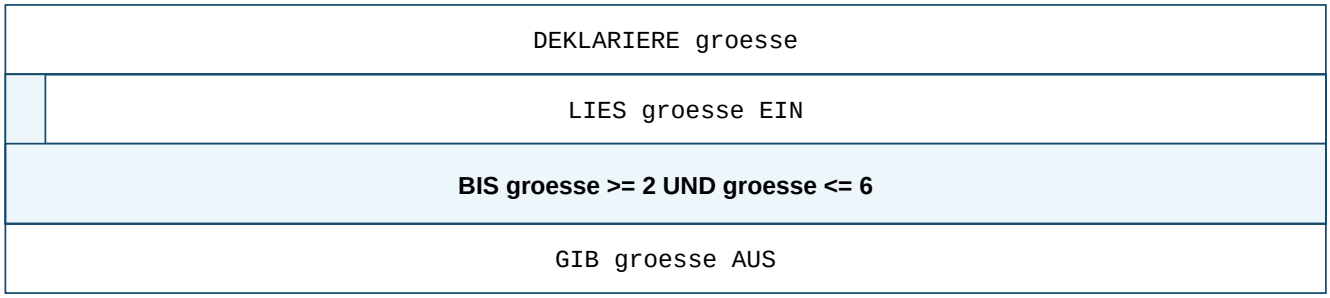
06 Eine gültige Gruppengröße einlesen

a) Gültigkeitsbedingung

`groesse >= 2 UND groesse <= 6`. Beide Grenzen sind eingeschlossen. ODER wäre hier falsch: Jede ganze Zahl würde mindestens eine der beiden Teilbedingungen erfüllen.

b) Pseudocode und Struktogramm

```
DEKLARIERE groesse
WIEDERHOLE:
    LIES groesse EIN
BIS groesse >= 2 UND groesse <= 6
GIB groesse AUS
```



c) Eingabefolge 0, 7, 4

Durchlauf	Eingelesene Größe	Gültig?	Folge
1	0	falsch	Noch einmal einlesen
2	7	falsch	Noch einmal einlesen
3	4	wahr	Schleife beenden

Ausgabe: 4. Die Variable wird erst nach dem Einlesen geprüft; ein künstlicher Startwert ist nicht erforderlich.

d) Sofort gültige Eingaben

Bei Eingabe 2 und bei Eingabe 6 gibt es jeweils genau einen Durchlauf und die entsprechende Ausgabe. Die Prüfung steht hinter der Eingabe. Ein erster Durchlauf findet deshalb immer statt. Die Schleife endet, sobald eine gültige Zahl eingegeben wird; eine ausschließlich ungültige Eingabefolge beendet sie nicht.

07 Bestellungen aufsummieren

Die Gesamtsumme startet bei 0. Für jede Bestellung bestimmt eine Selektion den Stückpreis. Danach werden der Bestellpreis und die neue Gesamtsumme berechnet.

a) Pseudocode

```
DEKLARIERE n, bestellung, anzahl, stueckpreis, preis, gesamt
LIES n EIN
SETZE bestellung AUF 1
SETZE gesamt AUF 0
SOLANGE bestellung <= n:
    LIES anzahl EIN
    WENN anzahl >= 10:
        SETZE stueckpreis AUF 2
    SONST:
        SETZE stueckpreis AUF 3
    ENDE WENN
    SETZE preis AUF anzahl * stueckpreis
    SETZE gesamt AUF gesamt + preis
    SETZE bestellung AUF bestellung + 1
ENDE SOLANGE
GIB gesamt AUS
```

c) Prüffälle

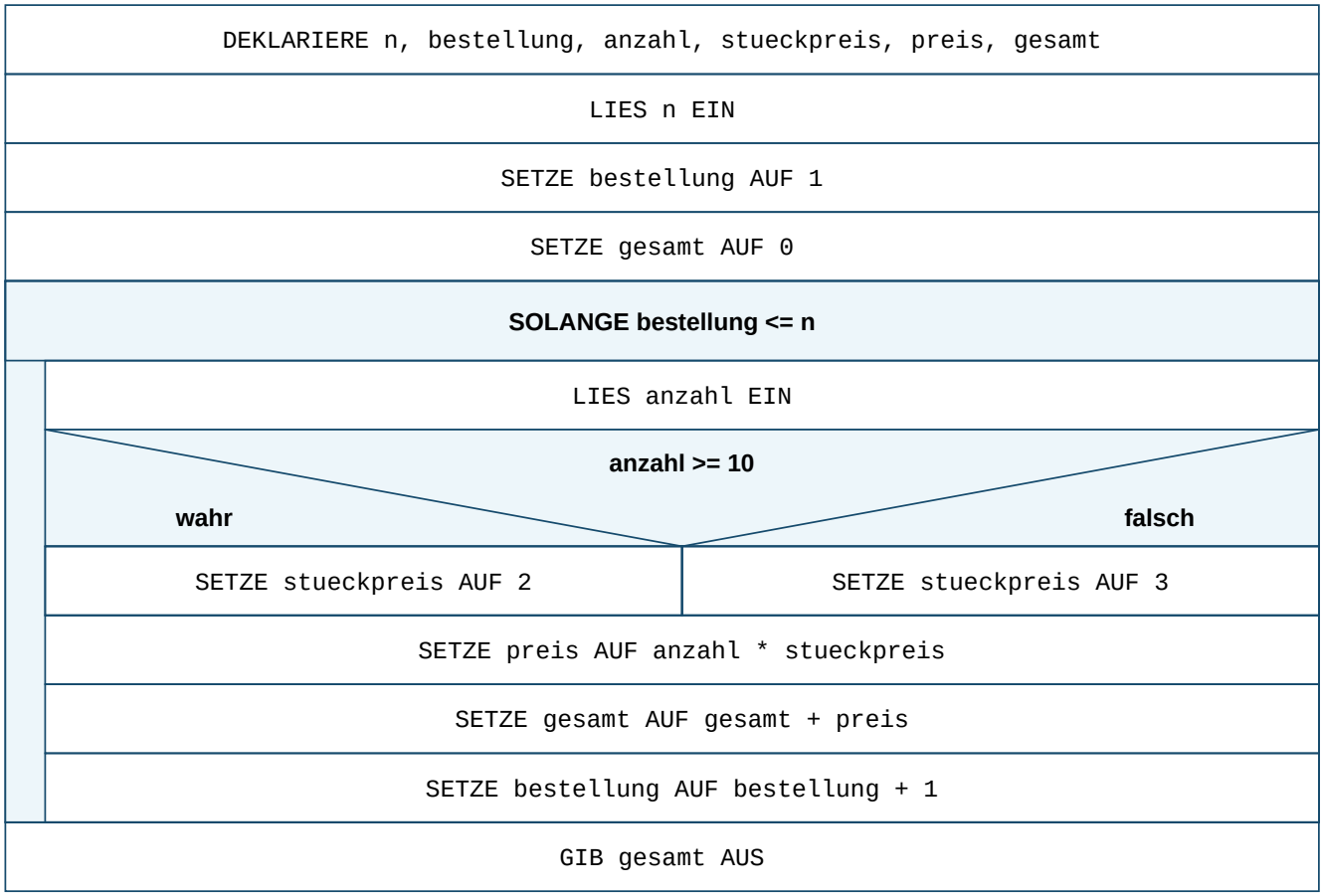
Bestellung	Anzahl	Stückpreis	Bestellpreis	Gesamtsumme
1	2	3 Euro	6 Euro	6 Euro
2	10	2 Euro	20 Euro	26 Euro
3	4	3 Euro	12 Euro	38 Euro

Ausgabe für n = 3: 38 Euro. Bei einer einzigen Bestellung mit 9 Heften beträgt die Ausgabe 27 Euro. Für n = 0 läuft der Schleifenrumpf keinmal und die Ausgabe ist 0 Euro.

Die Grenze 10 ist eingeschlossen. Der günstigere Stückpreis gilt für alle Hefte der jeweiligen Bestellung, nicht nur für Hefte ab Position 10.

07 Bestellungen aufsummieren (Fortsetzung)

b) Struktogramm mit verschachtelten Grundbausteinen



Sequenz: das gesamte Struktogramm mit Deklaration, Eingabe, Initialisierung, Schleife und Ausgabe.
Iteration: der umschließende SOLANGE-Block.
Selektion: die darin liegende Verzweigung `anzahl >= 10`.

Die Selektion wählt den Stückpreis. Die folgenden Berechnungen gehören zum Schleifenrumpf und laufen unabhängig davon, welcher Zweig ausgewählt wurde.

08 Eine Schleife prüfen und korrigieren

a) Fehlerhafte Fassung für n = 4

Prüfung	zaehler	summe	zaehler < n
1	1	0	wahr
2	2	1	wahr
3	3	3	wahr
4	4	6	falsch

Die tatsächliche Ausgabe ist 6. Verlangt ist 10. Bei `zaehler = 4` endet die Schleife bereits, sodass die 4 fehlt.

b) Korrektur

Die fehlerhafte Anweisung `SOLANGE zaehler < n` muss hier `SOLANGE zaehler <= n` lauten. Die übrigen Anweisungen bleiben gleich.

```
DEKLARIERE n, zaehler, summe
LIES n EIN
SETZE zaehler AUF 1
SETZE summe AUF 0
SOLANGE zaehler <= n:
    SETZE summe AUF summe + zaehler
    SETZE zaehler AUF zaehler + 1
ENDE SOLANGE
GIB summe AUS
```

c) Prüffälle der korrigierten Fassung

n	Addierte Werte	Ausgabe
0	Keine Werte	0
1	1	1
4	1 + 2 + 3 + 4	10

Die vollständige Trace-Tabelle der korrigierten Fassung für n = 4 steht bei Aufgabe 5. Dort findet vor dem Ende noch der vierte Durchlauf statt.

09 Division durch wiederholtes Subtrahieren

`r` enthält den noch nicht aufgeteilten Rest und startet bei a. `q` zählt, wie oft `b` vollständig abgezogen werden konnte, und startet bei 0.

a) Pseudocode

```
DEKLARIERE a, b, q, r
LIES a EIN
LIES b EIN
SETZE q AUF 0
SETZE r AUF a
SOLANGE r >= b:
    SETZE r AUF r - b
    SETZE q AUF q + 1
ENDE SOLANGE
GIB q, r AUS
```

Struktogramm

DEKLARIERE a, b, q, r
LIES a EIN
LIES b EIN
SETZE q AUF 0
SETZE r AUF a
SOLANGE r >= b
SETZE r AUF r - b
SETZE q AUF q + 1
GIB q, r AUS

09 Division durch wiederholtes Subtrahieren (Fortsetzung)

b) Trace-Tabelle für a = 17 und b = 5

Die Eingaben a und b bleiben unverändert. Die Tabelle zeigt q und r jeweils vor der Schleifenprüfung.

Prüfung	q	r	r >= b
1	0	17	wahr
2	1	12	wahr
3	2	7	wahr
4	3	2	falsch

Ausgabe: q = 3 und r = 2. Probe: 17 = 3 * 5 + 2.

Eingabe (a, b)	Quotient q	Rest r	Probe
(3, 5)	0	3	3 = 0 * 5 + 3
(15, 5)	3	0	15 = 3 * 5 + 0
(0, 5)	0	0	0 = 0 * 5 + 0

c) Warum endet die Schleife, und weshalb stimmt das Ergebnis?

Fortschritt: b ist positiv. Jeder Durchlauf verringert r daher um mindestens 1. Die Subtraktion erfolgt nur, wenn r >= b ist. Somit bleibt r nichtnegativ und kann nicht unbegrenzt weiter verringert werden.

Beim Ende: Die Bedingung r >= b ist falsch. Daher gilt r < b. Zusammen mit r >= 0 ergibt sich der verlangte Bereich 0 <= r < b.

Zusammenhang der Werte: Vor der ersten Prüfung gilt a = 0 * b + a. Ein vollständiger Durchlauf zieht b von r ab und erhöht q um 1. Dadurch gilt auch vor der nächsten Prüfung wieder a = q * b + r. Am Ende erfüllen q und r also beide Anforderungen.

Das Produkt q * b wird hier nur zur Begründung und zur Probe benutzt. Der Algorithmus selbst rechnet ausschließlich mit Addition und Subtraktion.

10 Welches Team gewinnt?

Benötigt werden die aktuelle Teamnummer, die Runde, der zuletzt gelesene Punktwert, die Teilsumme des aktuellen Teams sowie die bisher beste Summe und zugehörige Teamnummer. Es müssen keine früheren Rundenergebnisse gespeichert werden.

a) Pseudocode

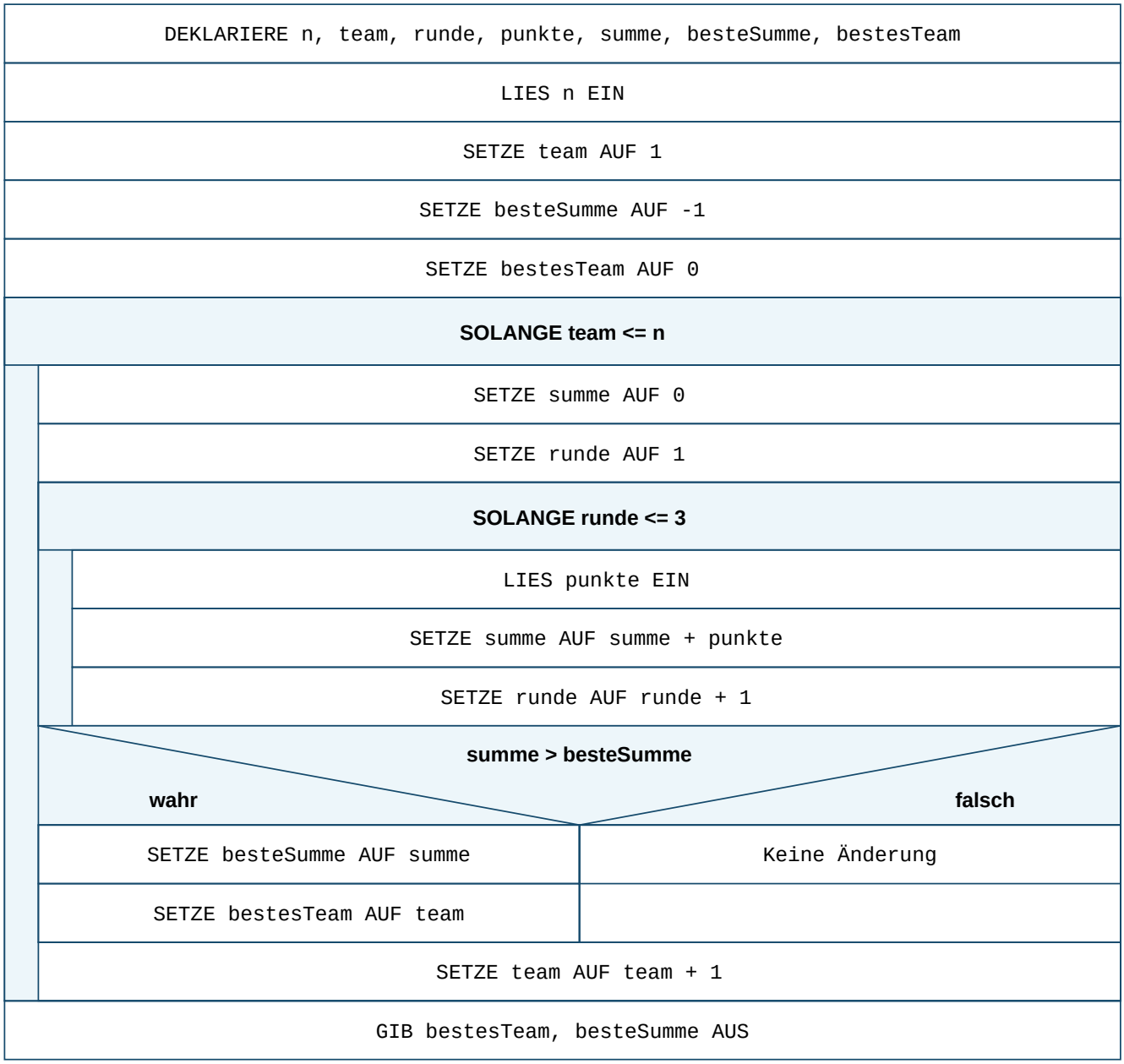
```
DEKLARIERE n, team, runde, punkte, summe, besteSumme, bestesTeam
LIES n EIN
SETZE team AUF 1
SETZE besteSumme AUF -1
SETZE bestesTeam AUF 0
SOLANGE team <= n:
    SETZE summe AUF 0
    SETZE runde AUF 1
    SOLANGE runde <= 3:
        LIES punkte EIN
        SETZE summe AUF summe + punkte
        SETZE runde AUF runde + 1
    ENDE SOLANGE
    WENN summe > besteSumme:
        SETZE besteSumme AUF summe
        SETZE bestesTeam AUF team
    SONST:
        TUE NICHTS
    ENDE WENN
    SETZE team AUF team + 1
ENDE SOLANGE
GIB bestesTeam, besteSumme AUS
```

`besteSumme = -1` liegt unter jeder gültigen Teamsumme. Daher übernimmt das erste Team sicher die Führung, auch bei 0 Punkten. `bestesTeam = 0` ist nur ein Startwert und wird vor der Ausgabe ersetzt.

`TUE NICHTS` bezeichnet einen leeren SONST-Zweig. Alternativ kann eine einseitige Selektion ohne SONST verwendet werden.

10 Welches Team gewinnt? (Fortsetzung)

b) Struktogramm



Zurücksetzen: `SETZE summe AUF 0` steht innerhalb der äußeren Schleife, vor der inneren. Die drei Runden bilden daher für jedes Team eine neue Teilsumme.

10 Welches Team gewinnt? (Fortsetzung)

c) Prüffälle

Team	Rundenpunkte	Teamsumme	Beste Summe danach	Sieger danach
1	4, 3, 2	9	9	1
2	2, 5, 2	9	9	1
3	1, 1, 1	3	9	1

Ausgabe: Team 1 mit 9 Punkten. Team 2 erreicht denselben Wert, ersetzt Team 1 jedoch nicht.

Fall	Eingaben je Team	Ausgabe
Ein Team, null Punkte	n = 1: (0, 0, 0)	Team 1, 0 Punkte
Letztes Team gewinnt	n = 3: (1, 1, 1), (2, 2, 2), (3, 3, 3)	Team 3, 9 Punkte
Nur null Punkte	n = 3: dreimal (0, 0, 0)	Team 1, 0 Punkte

d) Gleichstand und ausschließlich null Punkte

Gleichstand: Nur `summe > besteSumme` ersetzt den bisherigen Sieger. Bei Gleichheit bleibt er erhalten. Da die Teams aufsteigend von 1 bis n verarbeitet werden, hat der bisherige Sieger die kleinere Nummer.

Nur null Punkte: Team 1 übertrifft mit 0 Punkten den Anfangswert -1 und wird Sieger. Alle späteren Teams erreichen lediglich einen Gleichstand. Die Ausgabe ist daher Team 1 mit 0 Punkten.

Schleifen: Die äußere Iteration verarbeitet n Teams. Die innere verarbeitet für jedes Team genau drei Eingaben. Insgesamt werden 3 * n Punktwerte eingelesen. Sowohl die Teamnummer als auch der Rundenzähler werden jeweils um 1 erhöht.

Die Lösung kombiniert eine äußere Sequenz, zwei verschachtelte Iterationen und eine Selektion. Sie benötigt unabhängig von der Teamanzahl nur die sieben deklarierten Variablen.